

Day 2 - DSA Array 2 Pointers

ITSRUNTYM

What Is the Two Pointers Pattern?

The **Two Pointers** technique is a pattern where **two indices (pointers)** traverse a data structure (typically a **sorted array** or **string**) in a **coordinated way**, usually to avoid **$O(n^2)$** brute-force solutions and reduce them to **$O(n)$** or **$O(n \log n)$** .

Common Use-Cases:

- Finding pairs or triplets that satisfy a condition (like $\text{sum} = \text{target}$).
- Merging two sorted arrays or lists.
- Removing duplicates in-place.
- Reversing or partitioning an array.
- Comparing elements in two arrays.
- Sliding window optimization.

When to Use Two Pointers?

Use this pattern if:

1. **You're using two loops in brute force**, e.g., for every element, check all elements after it.
1. The **array is sorted** or **can be sorted** — this is key for many problems.
1. You're solving problems involving:
 - **Pairs** with a condition (e.g., $\text{sum} = \text{target}$)
 - **Windows** of data (e.g., longest substring, subarrays)
 - **In-place operations** (e.g., removing duplicates)

How It Works (Detailed Logic)

There are **two main styles** of Two Pointer usage:

1. Opposite Ends (Start and End)

- **Use case:** Finding a pair of numbers that sum to a target in a **sorted array**.
- **Logic:**
 - Start one pointer at the **beginning** (left) and the other at the **end** (right).
 - Move inward depending on the condition.

Example Problem:

Given a sorted array, find if there exists a pair of numbers that adds up to target.

2. Same Direction (Both start from left)

- **Use case:** Problems involving **sliding window** or comparing characters (e.g., substrings, subarrays).
- **Logic:**
 - Use two pointers like start and end to define a **window**.
 - Move the end to expand, and start to shrink the window as needed.

Example Problem:

[Remove Duplicates from Sorted Array](#)

Typical Steps in Two Pointer Approach

Let's formalize the steps:

1. Initialize Pointers

- left = 0, right = arr.length - 1 or both at 0, depending on the problem.

2. Iterate While Condition Is Valid

- Common conditions:
 - while (left < right)
 - while (right < n)
 - while (left <= right)

3. Check Condition

- Does the current window or pair satisfy the constraint?
 - $\text{sum} == \text{target}$
 - $\text{window length} \leq k$
 - characters are unique

4. Move Pointers

- Increase left or right depending on what you want:
 - Expand window
 - Shrink window
 - Explore next pair

5. Store or Update Result

- E.g., `maxLen`, `res.add(...)`, etc.

Visual Example

Array: [1, 2, 3, 4, 6], Target: 6

left = 0 (1), right = 4 (6)
→ $\text{sum} = 1+6 = 7 > 6$ → move right--
left = 0 (1), right = 3 (4)
→ $\text{sum} = 1+4 = 5 < 6$ → move left++
left = 1 (2), right = 3 (4)
→ $\text{sum} = 2+4 = 6$ //Found

#	Problem Name	Platform Link
1	Remove Duplicates from Sorted Array	LeetCode 26
2	Two Sum II - Input Array Is Sorted	LeetCode 167
3	Move Zeroes	LeetCode 283
4	Reverse String	LeetCode 344

5	Container With Most Water	LeetCode 11
6	Valid Palindrome	LeetCode 125
7	Squares of a Sorted Array	LeetCode 977
8	Subarray Product Less Than K	LeetCode 713
9	Remove Element	LeetCode 27
10	3Sum	LeetCode 15
11	Sort Colors (Dutch National Flag Problem)	LeetCode 75
12	Longest Substring Without Repeating Characters	LeetCode 3
13	Minimum Size Subarray Sum	LeetCode 209
14	Trapping Rain Water	LeetCode 42
15	Longest Mountain in Array	LeetCode 845